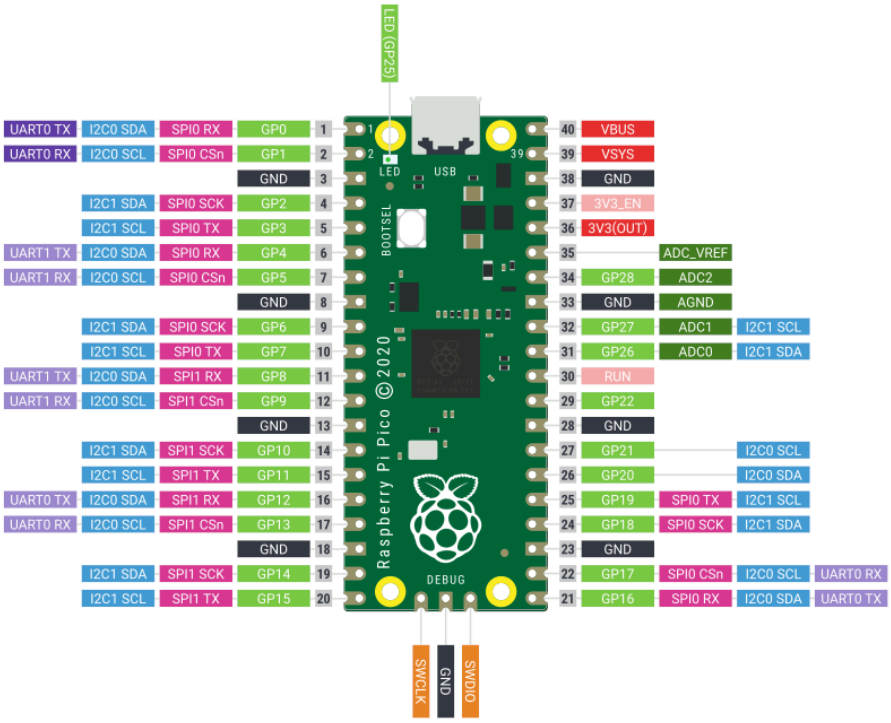
MicroPython jest implementacją języka Python 3, która może być uruchomiona w systemach wbudowanych takich, jak Raspberry Pi Pico lub Raspberry Pi Pico W. Przed rozpoczęciem pracy z modułem Raspberry Pi Pico należy zapoznać się z instrukcją *Drag-and-Drop MicroPython*, która jest dostępna pod linkiem:

www.raspberrypi.com/documentation/microcontrollers/micropython.html



Zad 1. Napisać program, który co pół sekundy będzie zmieniał stan LED na płytce Raspberry Pi Pico. Dioda elektroluminescencyjna jest podłączona do GPIO 25.

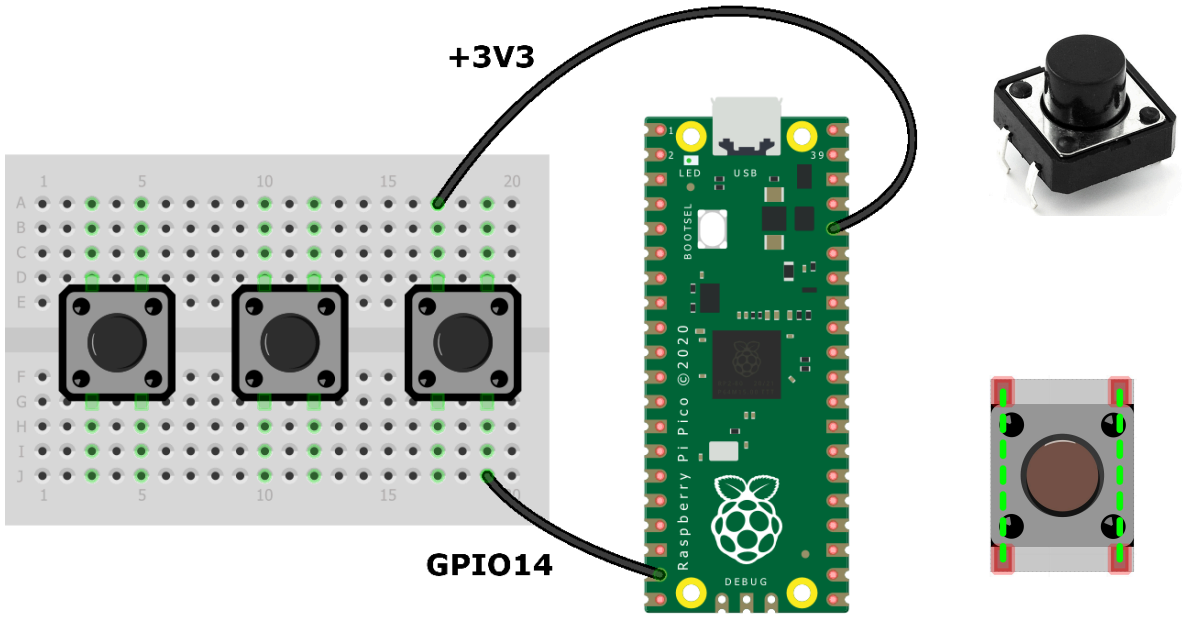
```
import machine
import utime as t

led_builtin = machine.Pin('LED',machine.Pin.OUT)
#led_builtin = machine.Pin(25,machine.Pin.OUT)

i = 0
```

```
while True:
      led_builtin.toggle()
      t.sleep(0.5)
      i+=1
      print(f'Current interation is {i}')
```

Zad 2. Połączyć układ jak na rysunku. Przycisk jest podłączony do GPIO 14 (GP14). Napisać program, który wyśle komunikat *"Button pressed"* do komputera po naciśnięciu przycisku:




```
from machine import Pin
import utime

button = Pin(14,Pin.IN,Pin.PULL_DOWN)

while True:
    if button.value() == 1:
        print("Button pressed")
        utime.sleep(0.5)
```

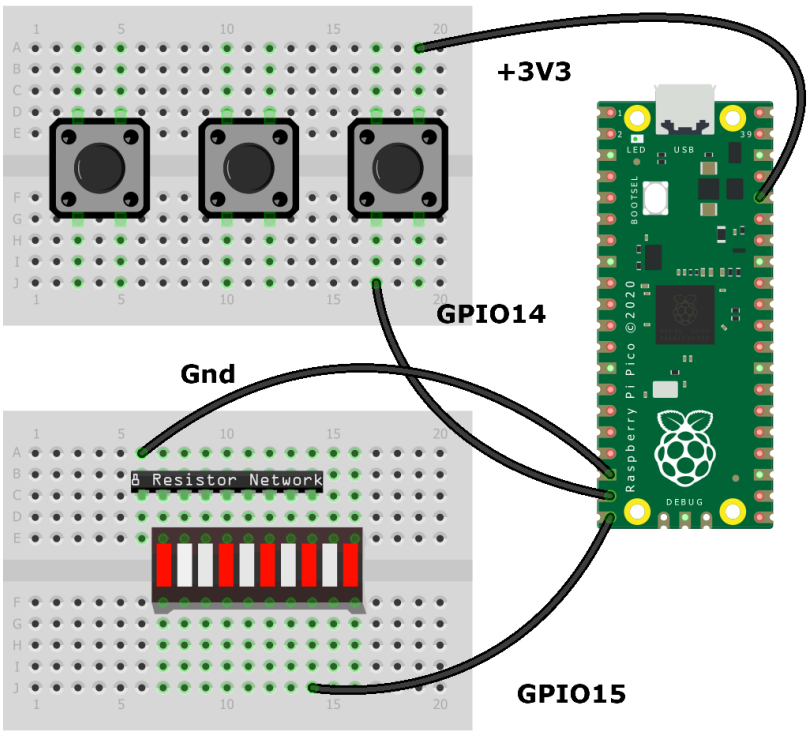
Pozwól użytkownikowi na podjęcie decyzji o treści komunikatu.

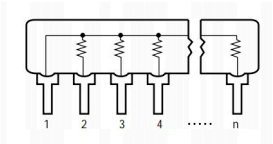
```
from machine import Pin
from utime import sleep

button = Pin(14,Pin.IN,Pin.PULL_DOWN)
message = input("Enter the message: ")
print('This is the message: ' + message)

while True:
    if button.value() == 1:
        print(message)
        sleep(0.5)
```

Zad 3. Połączyć układ jak na rysunku.





Jeżeli przycisk jest naciśnięty, LED na płytce powinna być wyłączona. W przeciwnym wypadku LED powinna być włączona. Drabinka rezystorowa jest elementem, który zawiera w swojej strukturze 8 rezystorów, które posiadają 1 z wyprowadzeń połączone ze sobą.

```
from machine import Pin
import utime

button = Pin(14,Pin.IN,Pin.PULL_DOWN)
led = Pin(15,Pin.OUT)
led.value(1)

while True:
    if button.value() == 1:
        led.value(0)
    else:
        led.value(1)
    utime.sleep(0.1)
```

Zad 4. Korzystając z układu z zadania 3 napisać program, który pozwoli na wygenerowanie 4 impulsów światła po każdym naciśnięciu przycisku.

```
from machine import Pin
from utime import sleep

button = Pin(14,Pin.IN,Pin.PULL_DOWN)
led = Pin(15,Pin.OUT)

def led_blink(pin,times):
    for i in range(times):
        pin.toggle()
        sleep(0.5)
        pin.toggle()
        sleep(0.5)
led.value(0)
```

```
while True:
    if button.value() == 1:
        led_blink(led,5)
    sleep(0.1)
```

Zad 5. Korzystając z układu, z zadania 3 napisać program implementujący funkcjonalność typowego włącznika oświetlenia z przyciskiem monostabilnym.

```
from machine import Pin
from utime import sleep

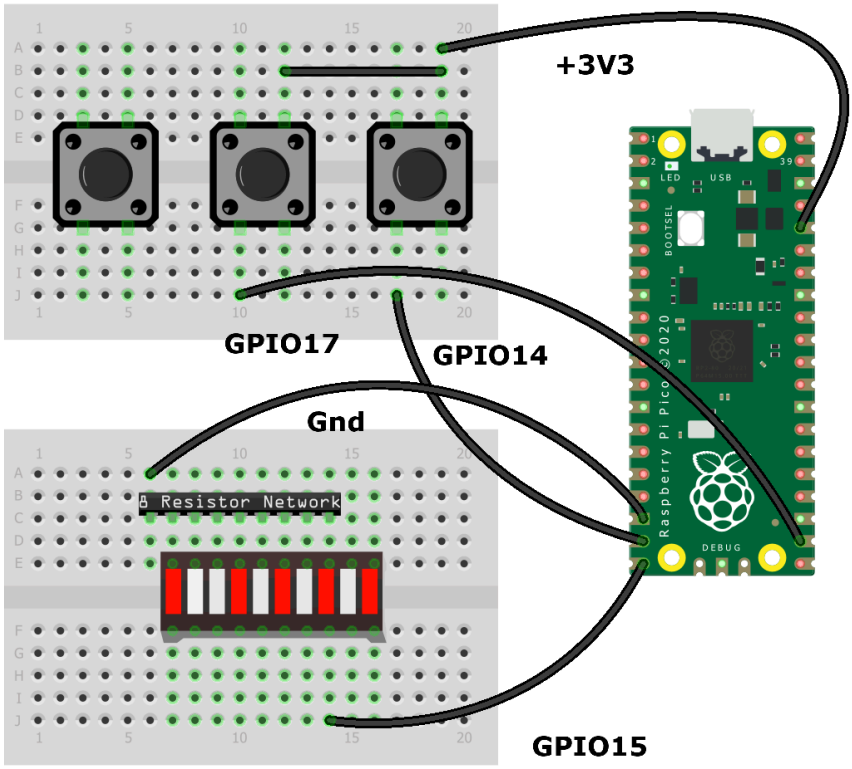
button = Pin(14,Pin.IN,Pin.PULL_DOWN)
light = Pin(15,Pin.OUT)

def button_clicked(pin):
    if pin.value() == 1:
        sleep(0.1)
        while pin.value() == 1:
            pass
        return True
    else:
        return False

light.value(0)

while True:
    if button_clicked(button):
        light.toggle()
```

Zad 6. Połączyć układ jak na rysunku. 1 z przycisków powinien załączać podłączoną do płytki Raspberry Pi Pico LED, a 2 wyłączać. Dioda na płytce powinna zmieniać swój stan 1 raz na sekundę.




```
from machine import Pin
from utime import sleep
import _thread

button_L = Pin(17,Pin.IN,Pin.PULL_DOWN)
button_R = Pin(14,Pin.IN,Pin.PULL_DOWN)
buildin_led = Pin(25,Pin.OUT)
external_led = Pin(15,Pin.OUT)
def led_blink():
    while True:
        buildin_led.toggle()
        sleep(0.5)
_thread.start_new_thread(led_blink,())
```

```
def button_clicked(pin):
    if pin.value() == 1:
        sleep(0.1)
        while pin.value() == 1:
            pass
        return True
    else:
        return False

while True:
    if button_clicked(button_R):
        external_led.value(1)
        print("R click")
    if button_clicked(button_L):
        external_led.value(0)
        print("L click")
```

Save this project in the memory of a Raspberry Pi Pico board as `main.py`. Disconnect and then reconnect the USB cable. Observe results.

Zad 7. Komunikacja pomiędzy wątkami.

```
from utime import sleep
import _thread

def core0():
    global core1_takes_control
    counter = 0
    while True:
        for _ in range(3):
            print(f' Counter value = {counter}')
            counter += 1
            sleep(1)

        core1_takes_control = True

        print("Core 0 is waiting")
```

```
        while core1_takes_control:
            pass

def core1():
    global core1_takes_control
    counter = 100
    while True:
        print("Core 1 is waiting")
        while not core1_takes_control:
            pass

        for _ in range(3):
            print(f'Counter value = {counter}')
            counter += 10
            sleep(2)

        core1_takes_control = False


core1_takes_control = False

second_thread = _thread.start_new_thread(core1,())
core0()
```

Zad 8. Współdzielenie zasobów.

Comment `lock.acquire()` and `lock.release()`.

```
from utime import sleep
import _thread

def core0():
    global lock
    while True:
        lock.acquire()
        print('C')
        sleep(0.5)
        print('O')
```

```
        sleep(0.5)
        print('R')
        sleep(0.5)
        print('E')
        sleep(0.5)
        print('0')
        sleep(0.5)
        lock.release()

def core1():
    global lock
    while True:
        lock.acquire()
        print('c')
        sleep(0.5)
        print('o')
        sleep(0.5)
        print('r')
        sleep(0.5)
        print('e')
        sleep(0.5)
        print('1')
        sleep(0.5)
        lock.release()

lock = _thread.allocate_lock()

_thread.start_new_thread(core1, ())
core0()
```

Zad 9. Polling the lock.

```
lock.acquire(True,1)
```

Parametry:

a. *blocking* (opcjonalne):
   - `True` (default) - wątek czeka, aż blokada będzie zwolniona.
   - `False` - wątek nie czeka - przechwytuję blokadę, jeżeli jest dostępna; jeżeli nie jest dostępna, wykonuję dostępny kod.

b. *timeout* (opcjonalny) - maksymalny czas (wyrażony w sekundach) oczekiwania na zwolnienie blokady.
   - -1 (default) - czas oczekiwania nieskończony.

Zawracane wartości:

- `True` - przechwycenie blokady.
- `False` - blokada nie została przechwycona (blocking=`False` lub timeout).

```
from utime import sleep
import _thread

def core0():
    global lock
    while True:
        wait_counter = 0
        while not lock.acquire(False):
#         while not lock.acquire(True,1):
            wait_counter += 1
             print(f'Core0 counted to {wait_counter} while
waiting')

        print('C')
        sleep(0.5)
```

```
        print('O')
        sleep(0.5)
        print('R')
        sleep(0.5)
        print('E')
        sleep(0.5)
        print('0')
        sleep(0.5)
        lock.release()

def core1():
    global lock
    while True:
        lock.acquire()
        print('c')
        sleep(0.5)
        print('o')
        sleep(0.5)
        print('r')
        sleep(0.5)
        print('e')
        sleep(0.5)
        print('1')
        sleep(0.5)
        lock.release()

lock = _thread.allocate_lock()

_thread.start_new_thread(core1, ())
core0()
```

Zad 10. (**0.5 punktu**) Podłączyć trzeci przycisk do płytki. Samodzielnie napisać program, który pozwala na załączanie i wyłączanie 3 wybranych segmentów. Załączenie/wyłączenie 1 segmentu powinno nastąpić 1 sekundę po naciśnięciu przycisku. 2 przycisk załącza/wyłącza 2 segmenty 2 sekundy

po naciśnięciu przycisku. 3 przycisk załącza/wyłącza 3 segmenty 3 sekundy po naciśnięciu przycisku.

Zad 11. (1 punkt) Napisać program, który pozwala na sterowanie położeniem jednego załączonego segmentu. Użytkownik musi mieć możliwość ustalenia, który segment jest załączany na początku programu.

Zad 12. (1.5 punktu) Zbudować prototyp urządzenia do pomiaru czasu reakcji.

Dla zainteresowanych:

1. Strona MicroPython:
   micropython.org/download/rp2-pico/

2. Python wprowadzenie:
   www.programiz.com/python-programming/first-program

3. Wprowadzenie do Raspberry Pi Pico:
   projects.raspberrypi.org/en/projects/getting-started-with-the-pico/0